Layouts and Rendering in Rails

1 Overview: How the Pieces Fit Together

This guide focuses on the interaction between Controller and View in the Model-View-Controller triangle. As you know, the Controller is responsible for orchestrating the whole process of handling a request in Rails, though it normally hands off any heavy code to the Model. But then, when it's time to send a response back to the user, the Controller hands things off to the View. It's that handoff that is the subject of this guide.

In broad strokes, this involves deciding what should be sent as the response and calling an appropriate method to create that response. If the response is a full-blown view, Rails also does some extra work to wrap the view in a layout and possibly to pull in partial views. You'll see all of those paths later in this guide.

2 Creating Responses

From the controller's point of view, there are three ways to create an HTTP response:

- Call `render` to create a full response to send back to the browser
- Call `redirect_to` to send an HTTP redirect status code to the browser
- Call `head` to create a response consisting solely of HTTP headers to send back to the browser

2.1 Rendering by Default: Convention Over Configuration in Action

You've heard that Rails promotes "convention over configuration". Default rendering is an excellent example of this. By default, controllers in Rails automatically render views with names that correspond to valid routes. For example, if you have this code in your `BooksController` class:

```
class BooksController < ApplicationController
end
```

And the following in your routes file:

```
resources :books
```

And you have a view file `app/views/books/index.html.erb`:

```
<h1>Books are coming soon!</h1>
```

Rails will automatically render `app/views/books/index.html.erb` when you navigate to `/books`and you will see "Books are coming soon!" on your screen.

However a coming soon screen is only minimally useful, so you will soon create your `Book` model and add the index action to `BooksController`:

```
class BooksController < ApplicationController
  def index
    @books = Book.all
  end
end
```

Note that we don't have explicit render at the end of the index action in accordance with "convention over configuration" principle. The rule is that if you do not explicitly render something at the end of a controller action, Rails will automatically look for the `action_name.html.erb` template in the controller's view path and render it. So in this case, Rails will render the `app/views/books/index.html.erb` file.